```
"""Velora Studio - Creator Finder.

Button dabane par GitHub isay chalata hai. Yeh:
  1) TikTok Creative Center (USA/UK) se trending creators aur trending videos ke
  2) followers ki had (maslan 2 lakh - 40 lakh) ke mutabiq chhaant'ta hai
  3) har creator ka naam / handle YouTube par check karta hai
  4) natija finder_state.json mein likhta hai (Velora app ka Finder tab wahi dikh

Yeh sirf dhoondta aur list banata hai. Kisi ki video download ya upload nahi kart
"""
import os
import re
import json
import time
import difflib
import datetime
import traceback
from zoneinfo import ZoneInfo

import requests

import ytpost  # Google login aur password wala lock yahin se

ROOT = os.path.dirname(os.path.abspath(__file__))
CONFIG_PATH = os.path.join(ROOT, "yt_config.json")
STATE_PATH = os.path.join(ROOT, "finder_state.json")
START = time.time()
BUDGET = 25 * 60
UA = ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
      "Chrome/124.0 Safari/537.36")

CC_PAGES = {
    "creators": "https://ads.tiktok.com/business/creativecenter/inspiration/popul
    "videos": "https://ads.tiktok.com/business/creativecenter/inspiration/popular
    "hashtags": "https://ads.tiktok.com/business/creativecenter/inspiration/popul
}


def load_json(path, default):
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return default
```

```
def time_left():
    return BUDGET - (time.time() - START)


def norm(s):
    return re.sub(r"[^a-z0-9]", "", (s or "").lower())


class Log:
    def __init__(self, state, tz):
        self.state, self.tz = state, tz

    def ts(self):
        return datetime.datetime.now(self.tz).strftime("%Y-%m-%d %H:%M")

    def __call__(self, msg, level="info"):
        print(f"[{self.ts()}] {level.upper()} {msg}", flush=True)
        self.state.setdefault("log", []).insert(0, {"t": self.ts(), "l": level, "
        del self.state["log"][120:]


# ---------------------------------------------------------------------- discovery
USER_RE = re.compile(r"tiktok\.com/@([A-Za-z0-9._]{2,40})")


def harvest(obj, region, out):
    """Creative Center ke JSON jawab mein se creators nikalta hai (format badle t
    if isinstance(obj, dict):
        user = ""
        for k in ("tt_link", "item_url", "share_url", "profile_url", "link"):
            m = USER_RE.search(str(obj.get(k) or ""))
            if m:
                user = m.group(1)
                break
        if not user:
            for k in ("unique_id", "uniqueId", "user_name", "username", "handle")
                v = str(obj.get(k) or "").strip().lstrip("@")
                if re.fullmatch(r"[A-Za-z0-9._]{2,40}", v):
                    user = v
                    break
        if user:
            followers = 0
            for k in ("follower_cnt", "follower_count", "followerCount", "fans_co
                try:
                    followers = int(obj.get(k) or 0)
```

```
                except (TypeError, ValueError):
                    followers = 0
                if followers:
                    break
            nick = ""
            for k in ("nick_name", "nickname", "nickName", "author_name", "creato
                if obj.get(k):
                    nick = str(obj[k])[:80]
                    break
            c = out.setdefault(user.lower(), {"username": user, "nickname": "", "
            c["nickname"] = c["nickname"] or nick
            c["followers"] = max(c["followers"], followers)
            if region and region not in c["regions"]:
                c["regions"].append(region)
        for v in obj.values():
            harvest(v, region, out)
    elif isinstance(obj, list):
        for v in obj:
            harvest(v, region, out)


DEFAULT_TOPICS = ["funny family videos", "mom life funny", "dad jokes", "funny ki
                  "family pranks", "grandma funny", "satisfying videos", "life ha
                  "pet funny", "skits comedy", "fails funny", "cleaning motivatio


def add_user(found, user, region):
    c = found.setdefault(user.lower(), {"username": user, "nickname": "", "follow
    if region and region not in c["regions"]:
        c["regions"].append(region)


def scrape_links(html, found, region):
    n = 0
    names = set(USER_RE.findall(html or ""))
    names |= set(re.findall(r'href="/@([A-Za-z0-9._]{2,40})', html or ""))
    names |= set(re.findall(r'"uniqueId":"([A-Za-z0-9._]{2,40})"', html or ""))
    for u in names:
        if u.lower() in ("tiktok", "discover", "music", "tag"):
            continue
        if u.lower() not in found:
            n += 1
        add_user(found, u, region)
    return n


def discover(regions, topics, log, state):
```

```
"""Do raaste: (1) TikTok Creative Center, (2) TikTok ke public 'discover' pag
from playwright.sync_api import sync_playwright
found, stats, diag = {}, {}, []
with sync_playwright() as p:
    browser = p.chromium.launch(args=["--no-sandbox", "--disable-blink-featur
    ctx = browser.new_context(user_agent=UA, locale="en-US", viewport={"width

    def open_page(url, region, label, api_words):
        page = ctx.new_page()
        got = {"n": 0, "api": 0}

        def on_resp(resp):
            try:
                if any(w in resp.url for w in api_words) and "json" in (resp.
                    got["api"] += 1
                    before = len(found)
                    harvest(resp.json(), region, found)
                    got["n"] += len(found) - before
            except Exception:
                pass
        page.on("response", on_resp)
        info = {"page": label}
        try:
            page.goto(url, wait_until="domcontentloaded", timeout=45000)
            page.wait_for_timeout(5000)
            for txt in ("Accept all", "Accept", "Allow all", "Got it"):
                try:
                    page.get_by_role("button", name=txt).first.click(timeout=
                    break
                except Exception:
                    pass
            page.keyboard.press("Escape")
            for _ in range(5):                                          # aur
                try:
                    page.locator("text=/View more/i").first.click(timeout=250
                    page.wait_for_timeout(2500)
                except Exception:
                    page.mouse.wheel(0, 5000)
                    page.wait_for_timeout(1500)
            got["n"] += scrape_links(page.content(), found, region)
            info.update(title=page.title()[:80], url=page.url[:120])
        except Exception as e:
            info["error"] = str(e).split("\n")[0][:120]
        info.update(api=got["api"], new=got["n"])
        diag.append(info)
        stats[label] = got["n"]
        page.close()
```

```
        for region in regions:
            for kind, tpl in CC_PAGES.items():
                if time_left() < 15 * 60:
                    break
                open_page(tpl.format(r=region), region, f"CC {region} {kind}", ["
        for t in topics:
            if time_left() < 12 * 60:
                break
            slug = re.sub(r"[^a-z0-9]+", "-", t.lower()).strip("-")
            open_page(f"https://www.tiktok.com/discover/{slug}", "", f"discover {
        browser.close()
    state["diag"] = diag
    bad = [d for d in diag if d.get("error")]
    if bad:
        log(f"{len(bad)} pages did not open, e.g. {bad[0]['page']}: {bad[0]['erro
    log("found: " + ", ".join(f"{k} {v}" for k, v in stats.items()))
    return found


def tiktok_profile(username):
    """TikTok profile page se followers aur naam (agar Creative Center ne na diye
    try:
        r = requests.get(f"https://www.tiktok.com/@{username}", headers={"User-Ag
                         timeout=25)
        if r.status_code != 200:
            return None, ""
        m = re.search(r'"followerCount":(\d+)', r.text)
        n = re.search(r'"nickname":"([^"]{1,80})"', r.text)
        nick = ""
        if n:
            try:
                nick = json.loads(f'"{n.group(1)}"')
            except Exception:
                nick = n.group(1)
        return (int(m.group(1)) if m else None), nick
    except Exception:
        return None, ""


# ---------------------------------------------------------------- YouTube chec
class YT:
    def __init__(self, cfg, key, log):
        self.log = log
        self.searches_left = int((cfg.get("finder") or {}).get("search_budget") o
        self.token = ""
        try:
```

```
            ctx = ytpost.Ctx(cfg, {})
            ctx.client_secret = (ytpost.decrypt_json(cfg["google_secret_enc"], ke
            ctx.refresh = {str(k): v for k, v in ytpost.decrypt_json(cfg["tokens_
            for ch in ctx.refresh:
                try:
                    self.token = ytpost.yt_access_token(ctx, ch)
                    break
                except Exception:
                    continue
        except Exception as e:
            log(f"Could not unlock Google login: {e}", "error")
        if not self.token:
            log("No working YouTube connection - connect a channel in the Channel

    def get(self, path, params):
        r = requests.get("https://www.googleapis.com/youtube/v3/" + path, params=
                         headers={"Authorization": f"Bearer {self.token}"}, timeo
        if r.status_code == 403 and "quota" in r.text.lower():
            raise RuntimeError("quota")
        r.raise_for_status()
        return r.json()

    def by_handle(self, username):
        d = self.get("channels", {"part": "snippet,statistics", "forHandle": "@"
        it = (d.get("items") or [None])[0]
        return it

    def search(self, q):
        self.searches_left -= 1
        d = self.get("search", {"part": "snippet", "type": "channel", "q": q, "ma
        return d.get("items") or []


def check(yt, c):
    """exists = YouTube par isi naam/handle ka channel hai, similar = milta julta
    user, nick = c["username"], c.get("nickname") or ""
    it = yt.by_handle(user)
    if it:
        s = it.get("snippet", {})
        return "exists", [{"title": s.get("title", ""), "url": f"https://www.yout
                           "subs": int(it.get("statistics", {}).get("subscriberCo
    if yt.searches_left <= 0:
        return "pending", []
    names = [user] + ([nick] if len(norm(nick)) >= 4 and norm(nick) != norm(user)
    matches, exact = [], False
    for item in yt.search(user):
        s = item.get("snippet", {})
```

```
        title = s.get("title") or s.get("channelTitle") or ""
        cid = (item.get("id") or {}).get("channelId") or s.get("channelId") or ""
        nt = norm(title)
        if not nt or not cid:
            continue
        is_exact = any(nt == norm(n) for n in names)
        ratio = max(difflib.SequenceMatcher(None, nt, norm(n)).ratio() for n in n
        contains = any(len(norm(n)) >= 5 and (norm(n) in nt or nt in norm(n)) for
        if is_exact or ratio >= 0.8 or contains:
            exact = exact or is_exact
            matches.append({"title": title, "url": f"https://www.youtube.com/chan
    if exact:
        return "exists", matches[:3]
    if matches:
        return "similar", matches[:3]
    return "not_found", []


# ---------------------------------------------------------------- main
def main():
    cfg = load_json(CONFIG_PATH, {})
    state = load_json(STATE_PATH, {})
    fcfg = cfg.get("finder") or {}
    tz = ZoneInfo(cfg.get("timezone") or "Asia/Karachi")
    log = Log(state, tz)
    regions = [r for r in (fcfg.get("regions") or ["US", "GB"]) if r in ("US", "G
    lo = int(fcfg.get("min_followers") or 200_000)
    hi = int(fcfg.get("max_followers") or 4_000_000)
    want = int(fcfg.get("max_results") or 30)
    creators = state.setdefault("creators", {})
    hidden = set(x.lower() for x in (cfg.get("finder_hidden") or []))
    state["running"] = True
    try:
        topics = fcfg.get("topics") or DEFAULT_TOPICS
        found = discover(regions, topics, log, state)
        # pehle se check kiye hue (aur 'pending' wale) bhi dobara line mein
        for k, c in creators.items():
            if c.get("status") == "pending" and k not in found:
                found[k] = {"username": c["username"], "nickname": c.get("nicknam
                            "followers": c.get("followers", 0), "regions": c.get(
        yt = YT(cfg, os.environ.get("APP_KEY", ""), log)
        done = new_green = lookups = 0
        queue = sorted(found.values(), key=lambda c: -int(c.get("followers") or 0
        for c in queue:
            key = c["username"].lower()
            old = creators.get(key)
            if key in hidden or (old and old.get("status") not in ("pending", Non
```

```
                continue
            if time_left() < 3 * 60 or done >= want * 3:
                break
            if not c.get("followers") or not c.get("nickname"):
                if lookups >= 250:
                    continue
                lookups += 1
                f, nick = tiktok_profile(c["username"])
                c["followers"] = c.get("followers") or f or 0
                c["nickname"] = c.get("nickname") or nick
            if c["followers"] and not (lo <= c["followers"] <= hi):
                continue
            if not yt.token:
                break
            try:
                status, matches = check(yt, c)
            except RuntimeError:
                log("Google search limit for today reached - the rest will be che
                break
            except Exception as e:
                log(f"@{c['username']}: YouTube check failed - {str(e)[:120]}", "
                continue
            creators[key] = {**c, "status": status, "matches": matches, "checked"
                             "tiktok": f"https://www.tiktok.com/@{c['username']}"
            done += 1
            new_green += status == "not_found"
        log(f"TikTok profiles looked up: {lookups}")
        log(f"done: {done} creators checked, {new_green} not found on YouTube"
            + (f", {yt.searches_left} searches left" if yt.token else ""), "succe
    except Exception as e:
        log(f"Finder error: {e} | {traceback.format_exc()[-300:]}", "error")
    # purane natije kam karo (aakhri 1500)
    if len(creators) > 1500:
        for k in sorted(creators, key=lambda k: creators[k].get("checked", ""))[:
            del creators[k]
    state["running"] = False
    state["last_run"] = log.ts()
    with open(STATE_PATH, "w", encoding="utf-8") as f:
        json.dump(state, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```